



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECJ1023

PROGRAMMING TECHNIQUE II

**GROUP PROJECT DELIVERABLE 2:
PROBLEM ANALYSIS AND DESIGN**

LECTURER:

DR JUMAIL BIN TALIBA

SUBMITTED BY:

DAYANG FARAH FARZANA BINTI ABANG IDHAM (A23CS0071)

FARRA NURZAHIN BINTI ZAHARIL ANUAR (A23CS0079)

NAWARRAH AUNI BINTI NAZRUDIN (A23CS0143)

SAFIYA NURSYAHADAH BINTI MASNOOR (A23CS0176)

SEMESTER II (2023/2024)

1.0 Introduction

We introduce "Fruit Blitz," an entertaining game with captivating concepts that aims to attract players. Our game is inspired by the popular game Fruit Ninja. In "Fruit Blitz," players' goal is to win points and get high scores by carefully avoiding bombs while slicing fruits.

In "Fruit Blitz", players use their computers' mouse to click on the fruits indicating the slice part through various fruits that appear on the screen, earning points for each successful slice. However, when bombs are placed among the fruits, the game becomes more difficult. In order to win the game, players must carefully avoid hitting the bombs, as doing so three times would result in loss. Additionally, by clicking on special fruit that can add up to additional points, players can enhance their point total. With its extra challenge, "Fruit Blitz" is an exciting and enjoyable game for players of all ages, guaranteeing that they stay focused and involved.

2.0 Problem Analysis

2.1 Objects and Classes

1. Game

This class manages the overall state and progression of a game. It keeps track of the current score and whether the game has ended. The game begins with a score of 0 and is not over until the endGame() method is called.

Attributes:

- currentScore: int = 0: The current score of the game, initially set to 0.
- isGameOver: bool = false: A flag indicating whether the game is over, initially set to false.

Methods:

- startGame(): void: Starts the game.
- endGame(): void: Ends the game.
- displayScore(): void: Displays the current score.

2. Player

This class represents a player in the game. It keeps track of the player's status including their lives, score and actions they can perform.

Attributes:

- name: String : The name of the player.
- lives: int = 3 : The number of lives the player has, initially set to 3.
- score: Score : An instance of the Score class to track the player's score.

Methods:

- sliceObject(object: SliceableObject) : void : Allows the player to slice an object.
- loseLife(): void : Decreases the player's life by one.
- isAlive(): bool : Checks if the player is still alive based on their remaining lives.

3. Score

This class manages the point system within the game. It handles the addition of points, resetting the score , and retrieving the high score. It ensures that the points system is maintained consistently throughout the game.

Attributes:

- points: int = 0 : The current points of the player initially set to 0.
- multiplier: float = 3.0 : A multiplier for calculating the points under normal conditions.
- multiplierSpecial: float = 5.0 : A special multiplier for calculating the points under special conditions.
- highScore: int : The highest score achieved in the game.

Methods :

- addPoints(points: int) : void : Adds points to the current score.
- resetScore(): void : Resets the current score to 0.
- getHighScore(): int : Returns the highest score achieved.

4. SliceableObject

This class represents objects in the game that can be sliced by the player. Each object has a position and speed in the game space and a flag indicating if it has been sliced. The object's state and behavior are managed through its attributes and interactions within the game environment.

Attributes:

- position: (float, float): The position of the object in the game space.
- speed: (float, float): The speed of the object.
- isSliced: bool = false: A flag indicating whether the object has been sliced, initially set to false.

Methods:

- move(timeDelta: float): void: Moves the object based on its velocity and the time delta.
- slice(): void: Slices the object.
- checkCollision(player: Player): bool: Checks if the object collides with the player.

5. Bomb

This class represents a sliceable object in the game. Its main role is to introduce an element of danger for the player. When a bomb is sliced, it can cause damage to the player, causing their lives to decrease.

Attributes:

- radius: float: The explosion radius of the bomb.
- damage: int: The damage caused by the bomb.

Methods:

- explode(): void: Causes the bomb to explode.
- causeDamage(player: Player): void: Causes damage to the player.

6. Fruit

This class represents a positive object in the game that the player should aim to slice. Slicing fruit typically rewards the plate with points, contributing to their score.

Attributes:

- type: string: The type of the fruit.
- points: int: The points awarded for slicing the fruit.

Methods:

- givePoints(player: Player): void: Awards points to the player.

2.2 Relationships

Inheritance

1. Bomb and Fruit with Sliceable Object

- Bomb and Fruit are subclasses of Sliceable Object. This means that both bombs and fruits share common properties and behaviors defined in Sliceable Object but also have their specific attributes and behaviors.
- To illustrate, fruits can be sliced for points, while bombs also can be sliced but should be avoided as they cause the player to lose lives or end the game.

Association

1. Player with Sliceable Object

The Player interacts with Sliceable Object instances. In the game, the player slices these objects by swiping on the screen.

Aggregation

1. Player with Game

The Game class aggregates Player objects. This means the game can have a player, but the player can exist independently of the game. In the game, a player might have a profile that keeps track of their high scores and other statistics.

2. Sliceable Object with Game

Game class manages a collection of SliceableObject instances (e.g., fruits, bombs). These objects exist independently of the Game instance, meaning they can be created, modified, and destroyed separately. This relationship allows the Game to interact with and manage the state of various sliceable objects

3. Player with Score

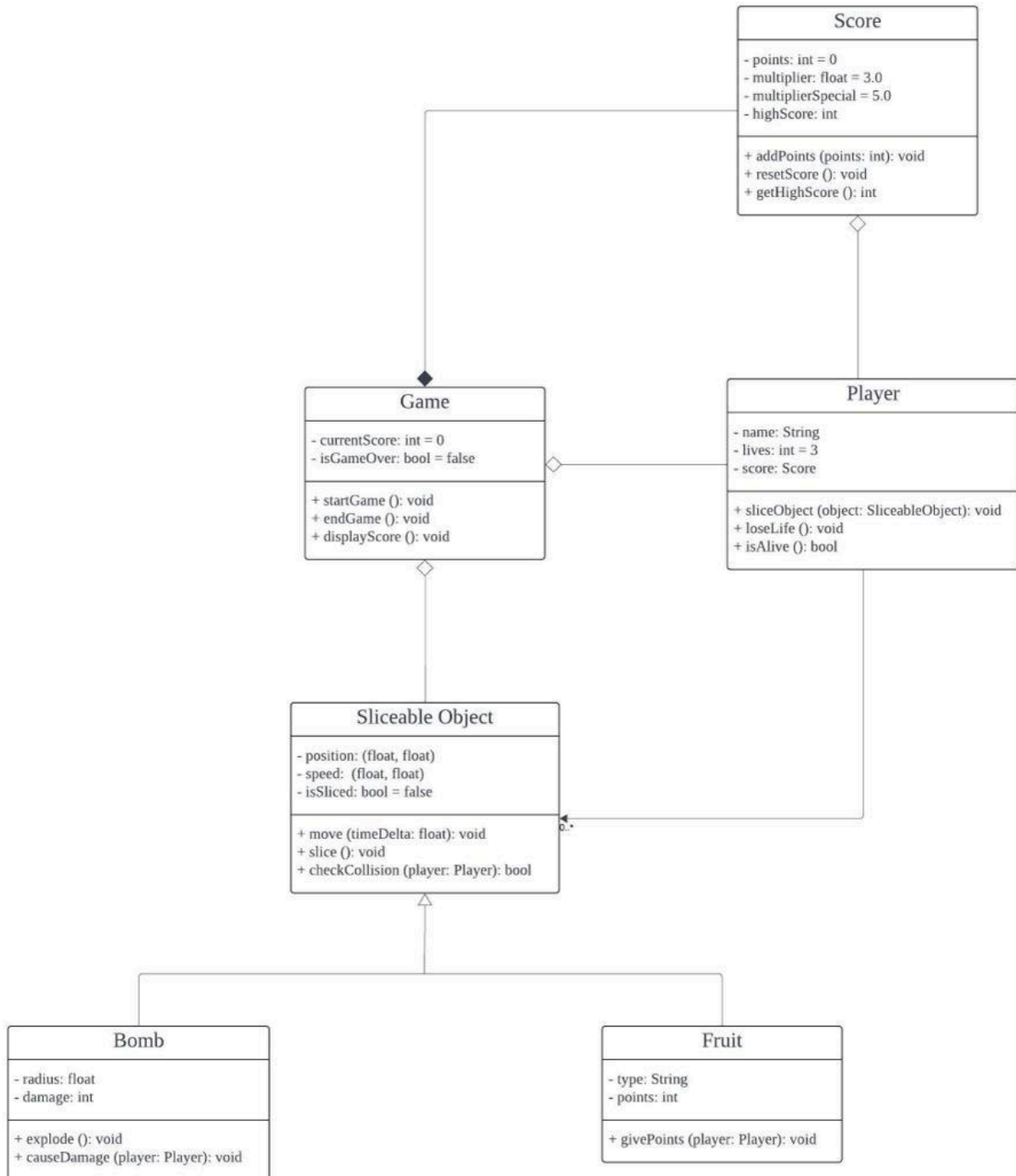
Aggregation between the Player and Score classes indicates that each Player object contains a Score object to manage the player's points. The Score instance exists independently of the Player. This relationship allows the Player to track and update their score through the Score object.

Composition

1. Score with Game

The Game class contains a Score object, meaning that the score is an essential part of the game. In the game, as you slice fruits, your score increases, and this score management is handled by the Score class.

3.0 Design



3.1 UML Diagram

4.0 Video Link

Project Deliverable 2 video link: <https://youtu.be/17xUdApouyM?feature=shared>